

目錄

問題簡述	p. 3
問題參數檔案配置	p. 3
問題對應解結構	p. 3
評估函式成本函數	p. 4
計算排程邏輯	p. 5
實驗環境	p. 5
演算法概要	p. 5
模擬退火	p. 6
基因演算法	p. 7 , 8
禁制搜尋	p. 9 , 10
啟發式方法	p. 11
啟發式混合 SA	p. 12
啟發式混合 TS	p. 12
基因混合禁制	p. 13 , 14
FOA 論文摘要	p. 15
FOA 連續型算法	p. 16
WOA 介紹	p. 17
離散 WOA	p. 18 , 19 , 20
DWOA+TS	p. 21
結果	p. 22 , 23
專案設計	p. 24 ~ 28
實驗專題結語	p. 29
個人心得	p. 30

問題簡述：

高斯消去法一種線性代數中常用的方法，用來解決矩陣方程系統的問題，多核處理器的使用日益普及，這帶來了如何有效地將高斯消去法與多核處理器的工作匹配與排程問題，其具有資料依賴性，但部分步驟可以同時進行。讓它成為**可並行優化的演算法**，適合在多核處理器上執行。要將高斯消去法分配到多核 CPU 上執行，就需要良好的任務分割與排程(scheduling)。在排程問題中，有一群任務，有多個資源處理器，每個任務有其屬性處理時間、前置任務，希望達成完成時間的最小化或最佳化目標，工作匹配與排程的組合最佳化問題，其高計算複雜度，難以通過窮舉完成，需用透過一些演算法設計來取得近似且品質可以接受的解。

問題參數配置：

在進行求解此問題，需要任務問題的基本資訊包含基本資訊、Communication Rate 通訊成本、其工作 任務分配至對應處理器的 Computation Cost、Data Transfer Volume 資料傳輸量。其中 處理器個數、工作個數、有向邊個數定義為整數資料，而 theCommRate、theCompCost、theTransDataVol 定義為二維陣列。在程式中進行配置的讀取，能夠解析 txt 檔案與 dag 檔案中問題案例之配置與資訊數量 數值，並存於對應安排的資料結構中，能夠依序讀取通訊速率矩陣、計算成本矩陣、傳輸資料表 並存在安排的資料結構中，對其進行管理，以便於後續的演算法使用。

此程式中，定義一個模塊資料結構用於存儲參數的配置，處理器 (Processor) 數量、任務 (Task) 數量、邊/傳輸量記錄數 (Edge Count)、處理器間通訊率矩陣 (PCount×PCount)、每個任務在各處理器上的運算成本 (TCount×PCount)、傳輸資料量清單 (ECount×3)，以及為了高效率的查詢與提取資料，用來記錄每個任務的前置任務及其資料量，可以通過任務標號的所有前驅任務及對應資料量的映射 predMap，使用雜湊表管理資料。

讀取配置檔案程式邏輯：

輸入串流開啟設定檔，對每一行進行檢查，根據標記 ID==1、3、5、7 決定接下來要讀取哪一段設定。每段前都呼叫 locate_to_section(infile, line);，將檔案指標移到該段的開頭。

- 讀取 P、T、E 三個計數 (ID==1)
- 讀取通訊率矩陣 (ID==3)
建立一個 P×P 的矩陣
- 讀取計算成本矩陣 (ID==5)
建立一個 T×P 的矩陣
- 讀取傳輸資料量清單 (ID==7)
每一列三個數值分別代表：
[來源任務編號, 目的任務編號, 資料量]。

建立 Hash Table 相鄰串列：

走訪剛剛讀進來的每一筆傳輸資料，將「to 任務」對應的「from 任務」和「資料量」放到 predMap[to] 的向量裡。這樣日後若要查某個任務的所有前置任務，就直接從 predMap[該任務] 讀即可。

最後回傳完整填好的 Config 物件。

問題對應解結構：

- 解的表示

以 class 定義，schedule string 定義一組可以容納 theTCount 長度一維整數資料，每一個數字是獨立的，其代表對應編號任務執行的順序；match string 定義一組長度為 theTCount 的一維陣列，其每一個資料的值域為 1~thePCount，代表對應的工作任務編號所交給對應數值編號的處理器。並定義浮點數用於存取此解的 cost 即排程總製造時間 makespan。

- 排程後甘特圖資料表示：

Schedule Result 以 class 或 struct 定義，定義兩組長度為 theTCount 的陣列，其分別代表一為每一個編號任務所對應開始執行的時間，另一則是對應編號任務工作結束的時間，由此後續便可用於甘特圖表式，並表示出此問題想看到的最終解，並存下 makespan。

評估函數

主要負責對排程解 (Solution) 進行評估與必要的可行性調整，並最終計算出該解的 makespan (總完工時間)，將結果回填至 Solution 物件中。

Solution：待評估的解，包括：

- ss：任務執行順序。
- ms：每個任務對應之處理器編號。
- cost：解的評估結果 (最後回填)。

ScheduleResult：評估後的中介結果，包含每個任務的 startTime、endTime 以及整體 makespan。

-- 主要函式說明：

Calculate_schedule(ss, ms, config)

根據給定的任務順序 ss 與處理器映射 ms，以策略計算每個任務的開始與結束時間，並返回對應的 makespan。

is_feasible(result, config)

檢查評估結果是否符合任務依賴 (precedence) 約束，即每條「來源→目的」邊的來源任務必須先完成。對 config.theTransDataVol 所有邊逐一驗證 $endTime[from] + commCost \leq startTime[to]$ ；若違反可選擇列印錯誤訊息，並回傳 false。

Solution_Function(sol, config)

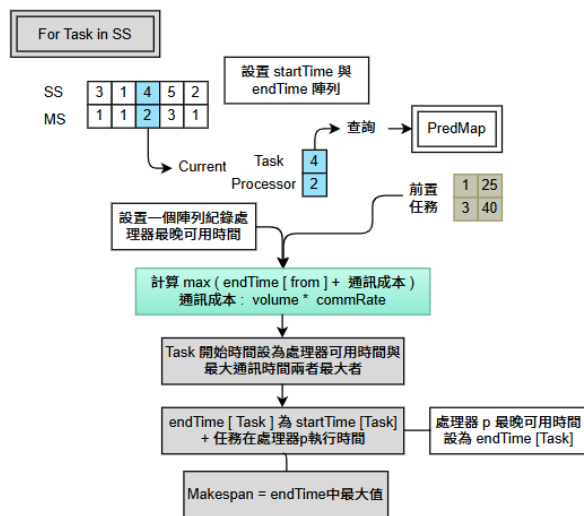
整合前兩者，先檢查解的基本合法性，再呼叫 Calculate_schedule 取得結果，最後在發現依賴違規時，依據拓譜邏輯約束排序其解並改成可行解，並重新計算，直到通過可行性檢驗為止。最終將 makespan 存回 sol.cost 並回傳最終 ScheduleResult。

邏輯流程說明

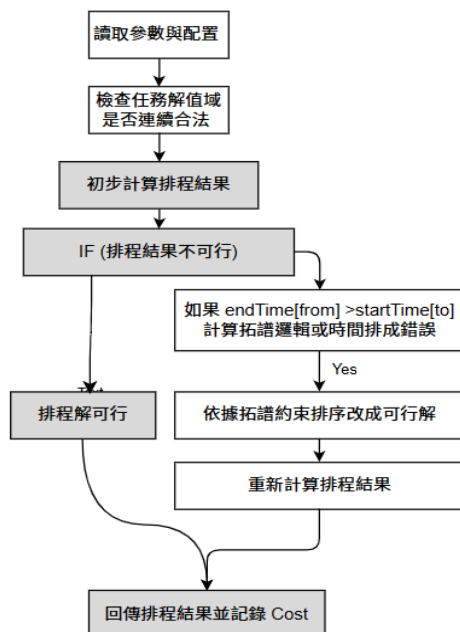
- 傳入排程解與問題配置
- 初始化
 - P 長度陣列紀錄所有處理器可用時間。
 - startTime、endTime 初始化
- 初步檢核
 - 確認 $sol.ms.size() == T$ ，以及所有 $ms[i] \in [0, P)$
- 初次排程計算
 - $result = Calculate_schedule(sol.ss, sol.ms, config);$
 - 遍歷每一個排程好的任務
 - 取出當前任務、對應處理器
 - 計算依賴完成時間 ready
 - 若 t 有前驅任務，則遍歷每個 (from, vol)
 - 取出 from、vol 傳輸成本
 - 若 from 跟 t 處理器相同使成本 0
 - 否則 $vol * 對應 theCommRate$
 - ready 為其所有 edge 最晚結束者
 - 確定其處理器可用並安排至 ready 開始
 - 回傳排程結果並記錄 makespan
- 可行性檢驗與調整迴圈
 - 回到 Solution_Function，在取得初次 result 後
 - 對每條傳輸邊檢查若違反就調整
 - 若違反進行依照拓譜邏輯排序
 - 在排程計算結果
 - 直到所有依賴皆符合為止或無調整空間
 - 回傳最終 排程結果與紀錄 sol.cost

此處，分離計算與驗證，排程計算 (Calculate_schedule) 與可行性檢驗 (is_feasible)，並進行動態調整 採用違反即調整的策略，保證解序列逐步修正為可行解。最壞情況下，每次重新排程需 $O(T \cdot (P + E))$ ，調整迴圈次數 $\leq E$ ，適中可用於中小規模排程評估。涵蓋從初始檢核、ASAP 排程計算、依賴驗證，到動態調整。以此便可以將其模組化，並在後續使用元啟發演算法進行實驗時，作為成本函數或是適存值函數使用。

排程計算流程圖：



最初設計排程計算時是通過存取 Edges 並調用與查詢，再後續進行一些優化，改為使用 Hash Table 建立相鄰串列 predMap，以 Map 的方式進行運用查詢，Key 值以 To 來去映射到一個 pair value，內含有 From 跟 Volum，以「目標任務 → (前置任務, 資料量) 列表」存放，日後在排程時只要查 predMap[t] 就能快速拿到所有前置任務，取代遍歷全邊集的作法。查詢效率從 $O(E) \rightarrow O(1 + k)$ ：原本每次計算某任務的就緒時間，都要掃描 E 條邊；現在只查 predMap[to]，時間複雜度為該任務實際前置數 k。



實驗環境：

電腦計算機：LAPTOP-NC8UDS16
 處理器：12th Gen Intel(R) Core(TM) i7-12700H 2.70 GHz
 RAM：16.0 GB
 作業系統：Windows 11
 程式語言：C++ with STD lib
 其他工具：Python Matplotlib

問題案例程式化編碼說明：

問題案例之高斯消去，解空間編碼以離散空間為主，使用 c++ vector 運算，編碼方式由 1 ~ Tcount 與 1 ~ Pcount。在使用連續型演算法時，通過以浮點數陣列編碼，並在呼叫成本函數或適存值計算時，通過轉換映射的方式轉換到離散解空間，SS 部分使用將浮點數轉為排序位置 SPV Encoding，MS 部分通過數值等頻分組，將連續型變數轉為離散型分類。

使用演算法類型簡要

離散型演算法，此次實驗中使用之元啟發式演算法包含使用 模擬退火 Simulated Annealing、基因演算法 Genetic Algorithm、禁制搜尋 Tabu Search。

連續型演算法，則在此次中參考兩篇論文提出的新興元啟發算法，鯨魚優化演算法 Whale Optimization Algorithm、狐狸優化演算法 FOX Optimization Algorithm，兩者基於模仿生物狩獵與搜尋行為並建構數學模型在連續空間中探索解，為了將其應用在此次實驗實作中，我採用兩種做法使算法得以被運用在離散解空間中使用，一是利用排序大小映射排序字串並利用等頻分組方法映射至離散的解空間中，另一種作法則是通過參考現有文獻與其他離散操作運算子的元啟發式優化演算法做離散解擾動運算，將仿生數學模型改成利用離散運算子，像是交換、改變單點值、Order Crossover 或段落改動等方式，並在此次實驗利用在 WOA 演算法中進行離散變體。

模擬退火 Simulated Annealing

初步的實驗中，我採用簡易的模擬退火優化算法，通過設置溫度控制，通過離散操作擾動進行搜尋解空間，並以機率數學模型運算決定是否接受或改變現在的可行解。此處應用，以模擬退火演算法求解排程，排程時間 makespan 最小化問題，在程式中透過隨機改動資料方式與對每個任務隨機選擇一個處理器，以此生成初始解作為模擬退火算法之輸入，或是可以使用啟發式貪婪演算法生成初始解，或後續作為 Local Search。

• 初始溫度 (Initial Temperature)

控制初期接受較差解的機率，使搜尋具探索性，可跳脫區域最優。程式預設 100.0，或通過問題測試品質調整或可以進一步利用自適應參數調整。

• 最低溫度

當溫度下降到此值以下即停止演算法。

• 冷卻係數 (Cooling Rate)

每次溫度迴圈後做 $T *= \alpha$ 。設置介於 0.8–0.99 之間。

• inner loop iterations

每個溫度水平下會進行的鄰域搜尋次數。

演算法會進行迴圈迭代，總計設置將迭代 200 次，並且設置成若是多次迭代沒有改進會停機。參數的設置與調整，除了迭代次數固定為 200 次，依問題大小、鄰域成本計算速度、所需平衡探索與效率取適當值，或是後期加入自適應模組調整算法優化效果。

SA 鄰域搜索方式

演算法運行迭代過程中將會通過擾動、離散操作運算，改變現在可行解，進一步搜尋鄰域解空間中的更佳解或是進一步跳出局部搜尋，在全域搜索更佳解搜尋。主要思路為隨機選擇三種操作之一，令新解在原解附近做離散變動。

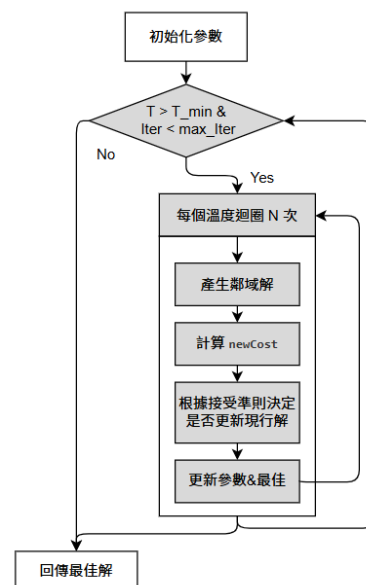
- Swap SS : 隨機挑取兩個索引交換
- Change MS 隨機挑取一索引改變數值
- Swap + Change 同時採用兩種

離散操作運算設計思路，單一操作有較小變動，適合細部局部搜尋；兩者同時改動幅度較大，有助跳出區域最優。

接受準則 (Acceptance Criterion)

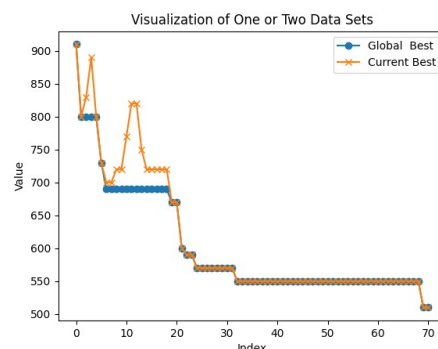
經典 SA 做法：計算 $\delta = \text{newCost} - \text{currentCost}$ 。若 $\delta \leq 0$ （新解成本不高於目前），則必定接受（accept = true），並更新 current_S, currentCost。若 $\delta > 0$ （新解較差），以機率 $\exp(-\delta / T)$ 接受，以 $\text{if} ((\text{double})\text{rand}() / \text{RAND_MAX} < \text{prob})$ 判斷是否接受。當接受後，若 $\text{newCost} < \text{best_Cost}$ ，更新全域最優。

迭代主流程：



測試結果

- 案例 0 Avg (100 次) : 557.9
- 案例 2 Avg (100 次) : 540.642
- 案例 4 Avg (100 次) : 535.774
- 案例 6 Avg (100 次) : 557.405



案例 n4_00 迭代曲線視覺化

基因演算法 Genetic Algorithm - 1

基因演算法的主要類型分成 世代制 GA、穩態 GA，此處先實驗 世代制 GA，每一代會從整個父代族群選擇並生成一整套後代，然後以後代完全或部分取代父代，然後進入下一代。每一個 generation，都先做 Selection 產生 mating_pool，再對整個族群大小次數做交配與突變，將子代收集到 next_pop，最後用世代交替完全取代。

染色體與個體

染色體編碼由解 ss 與 ms 組成，個體交配過程時，基因染色體會改動，需保持合法排列，交配操作需確保不產生重複或遺漏，匹配部分不超出值域。

此處，以個體 Individual 來表示一個解繼承自 Solution 編碼，並可以為每一個個體計算適存值，構造個體時以隨機生成一個解產生初始解，並計算適存值與排程結束時間。適存值為排程結束時間 Cost 倒數。

交配方式 (Crossover)

此處的 ss 交配運算子設計中，會通過隨機決定是否執行交配，對 ss 部分執行交配；否則子代直接繼承父母之一，若未交配則 child 保持與 this 相同的 ss。對 ss 交配方式，使用 OX (Order Crossover)，隨機選兩個剪切點，將父母 A 在區間的基因直接複製到子個體中對應部分，標記為已用，然後從另一個父母 B 區間結束點開始，依序掃描長度 T 的環狀序列，若該基因尚未使用，直到填滿所有位置。

對 ms 的交配，採用 Uniform Crossover 機率，對每個基因位 i，隨機以 0.5 機率從父母 1 或父母 2 繼承。

以此，生成子代個體，計算成本與 fitness，返回子個體。交配率設為 0.7。

突變方式 (Mutation)

使用機率方式決定是否對 ss 突變，ss 部分之突變方式，隨機選兩位置進行互換，ms 部分，則是隨機選索引 k，對其索引重新隨機分配機器。突變率設為 0.2 ~ 0.4。早期探索可用較高突變率，後期逐漸降低；或基於適應度變化自適應調整。

選擇方式 (Selection)

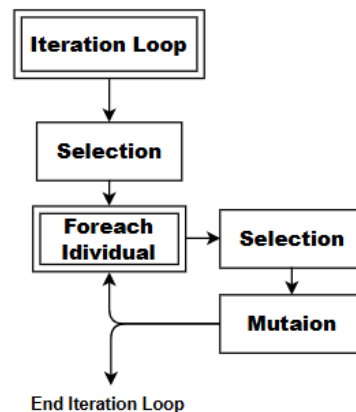
分成 競賽式選擇、輪盤式選擇，競賽式選擇每次從族群中隨機選取若干個體，在這些候選者中選擇適應度最高者作為父母。重複此程序直到獲得足夠父母數量。輪盤式選擇則根據個體適應度與族群總適應度的比例分配選擇機率，再對每次選擇生成隨機數決定落在哪個區間，即選擇該個體。

GA 主體演算法

參數：世代(迭代次數)、族群大小(個體數量)、交配率、突變率、選擇法。

初始族群生成 (Initialization)

世代演化：



Return 歷史最佳解

實驗結果：

族群大小: 20、迭代 200 次世代

Tournament Selection

- 案例 0 Avg (100 次) : **518.2**
- 案例 2 Avg (100 次) : **515.332**
- 案例 4 Avg (100 次) : **502.388**
- 案例 6 Avg (100 次) : **507.5**

Roulette Selection

- 案例 0 Avg (100 次) : 575.2
- 案例 2 Avg (100 次) : 572.305
- 案例 4 Avg (100 次) : 559.276
- 案例 6 Avg (100 次) : 582.566

在此可看出從 SA 到 GA，使用競賽式選擇有了很穩定的進步，可以更好找到品質好的解，實驗最佳情況可以做到 410 ~ 430 的 makespan。後續可再改良，可嘗試 PMX、CX、ERX 等排列交配，或是動態調整選擇方式、參數等。

基因演算法 Genetic Algorithm - 2

穩態 GA，每次產生少數後代（如 1~k）
→ 替換部分父代，部分替換：常見做法是替換最差個體或隨機替換，生成 single child 並替換最差，父母一與父母二則透過輪盤或是競賽輪盤選出，進行如先前的交配與突變，本身即保留大部分族群，此次程式算法實驗中，設置 GA 將會在每一次世代中生成族群大小除以 2 數量的子代，並取代最差者。此法相比傳統全體替換的 GA，在維持族群穩定與收斂控制上更具彈性與效率。Steady-State GA，每一代僅產生一位新子代或少數子代，使用 Tournament 或 Roulette Selection 選出兩位親代進行交配與突變，產生的子代若表現優於族群中最差個體，則替代之，族群維持恆定大小，每代更新當前最佳解，**並保留其不被替換**，以確保演化穩定性。

實驗結果：

族群大小: 20、迭代 200 次世代

Tournament Selection

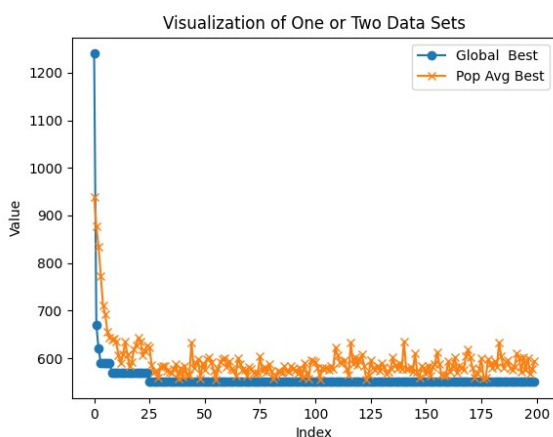
- 案例 0 Avg (100 次) : 545.2
- 案例 2 Avg (100 次) : 530.212
- 案例 4 Avg (100 次) : 513.954
- 案例 6 Avg (100 次) : 523.023

Roulette Selection

- 案例 0 Avg (100 次) : **532.6**
- 案例 2 Avg (100 次) : **523.882**
- 案例 4 Avg (100 次) : **500.705**
- 案例 6 Avg (100 次) : **521.626**

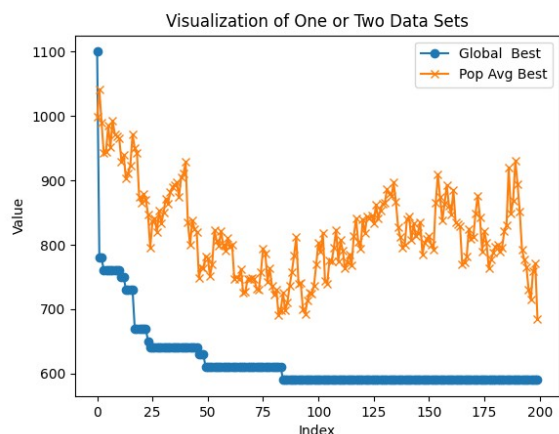
收斂曲線與視覺化迭代

GA1 Tournament Selection 收斂曲線



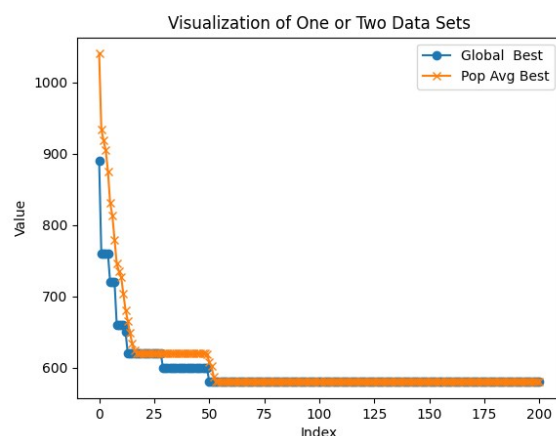
案例 n4_00 迭代曲線視覺化

GA1 Roulette Selection 收斂曲線



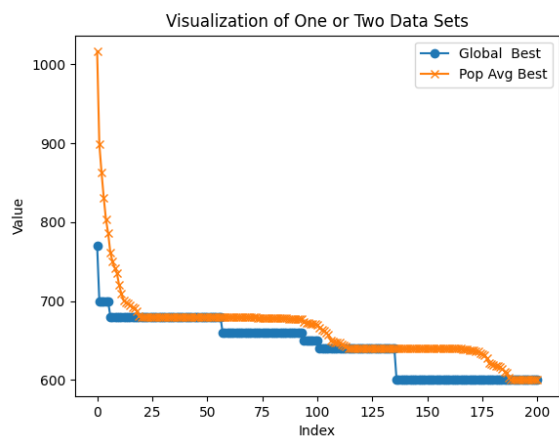
案例 n4_00 迭代曲線視覺化

GA2 Tournament Selection 收斂曲線



案例 n4_00 迭代曲線視覺化

GA2 Roulette Selection 收斂曲線



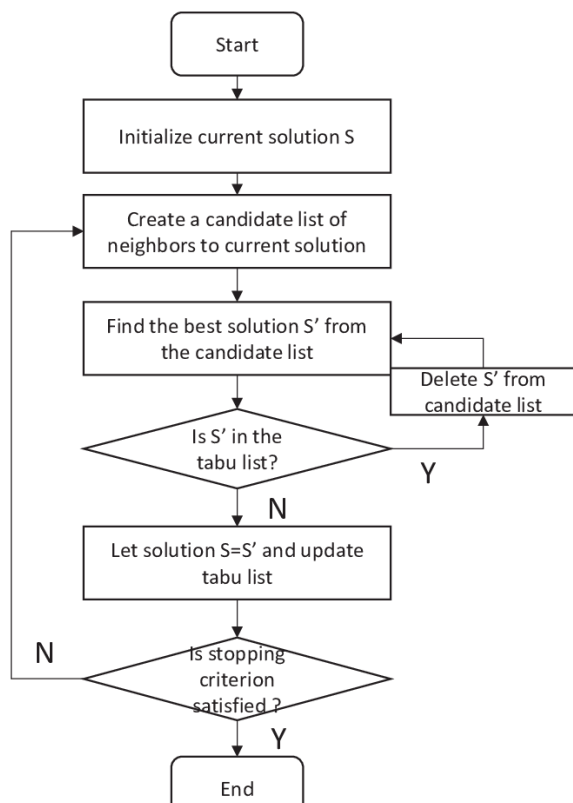
案例 n4_00 迭代曲線視覺化

Tabu Search

應用在此問題上 Tabu Search（禁忌搜索）是一種有效的演算法，用來解決工作匹配與排程問題，特別是在多核處理器上執行高計算複雜度的任務分配。從一個初始解出發，透過不斷移動到鄰近解，尋找更好的解。

對比 Hill Climbing 兩者都是**局部搜尋**，Tabu Search 可以跳出局部最優，探索更多潛在解空間，在遇到不再改善的情況下，可以嘗試調整至稍差的解，但利用禁忌表避免回到之前的狀態，從而有機會跳出局部最優、繼續搜尋更佳解。

引入概念，在局部搜索方式，加入記憶機制（禁忌表），容忍次優 可以搜尋廣域的解。



初始化會產生一個初始解，或是由 Greedy Heuristic、其他算法產生解或隨機產生，設定當前解為目前最佳解，初始化禁忌表（Tabu List）為空。

重複以下步驟 進行迭代優化，根據當前解產生一組鄰近解，篩選可行鄰近解，檢查這些鄰近解中，有哪些動作在禁忌表中被標記為禁止的，移除這些被禁止的選項，**但允許例外**：若某個解雖然是禁忌，但比目前最好的解還好，從剩下的鄰近解中選擇一個最優的解。將當前解更新為選中的鄰近解，如果該解比目前最佳解還好，更新為新的最佳解，把剛才進行的移動 加入禁忌表。若禁忌表已滿（超過預設長度），則移除最早進入的項目（即使用 Queue 的先進先出策略）。

鄰居搜索方式：

我的搜索鄰居區域方式，鄰居產生策略有兩種，每次隨機挑選其中一種來執行，讓搜尋過程保持多樣性、避免陷入局部最優。

- 交換兩個不同作業的位置（Swap）
從目前的任務序列中隨機選擇兩個不同的作業 i 與 j ，將這兩個作業在序列中的順序互換。
- 更換一個作業的機台
隨機選擇一個作業 t 。將其指定的處理機台 P_{old} 改為另一個不同的機台 P_{new}

隨機選擇一種策略（交換 or 機台更換）

IF 選擇「交換位置」

- 從作業列表中隨機選擇 $i \neq j$
- 交換作業 i, j 的順序
- 產生新解 $new_solution$
- 返回 $new_solution$

ELSE IF 選擇「更換機台」

- 隨機選擇一個作業 t
- 從可用機台列表中選擇與目前不同的 $newP$
- 將作業 t 分配到 $newP$
- 檢查是否可行（符合機台限制）
- 產生新解 $new_solution$
- 返回 $new_solution$

Move 的定義：

交換兩個作業在排程中的順序，或是更換某一作業的處理機台，需記錄欄位包含，作業 i、作業 j，作業 t、舊機台、新機台。同一個 Move 結構中便會記錄搜索鄰居時的動作，根據策略 1 或 2 執行動作後，就可以建立一個對應的 Move 實例。

```
MoveType type;
int i, j;
int t, old_P, new_P;
```

Tabu List：

基於 Queue 設置 Tabu List。

add(const Move&)

禁忌清單的管理邏輯，包括加入禁忌、查詢是否禁忌、以及定期衰減禁忌期限。將某個 Move 加入禁忌清單中，並設下禁忌期限 maxTenure，如果已存在相同 Move，則更新其期限為最新。執行完某個 Move（交換或機台更換）後，把它記進 Tabu List，未來數輪內禁止做相同操作。

contains(const Move&)：

在產生鄰居後過濾，這個動作在禁忌清單內嗎？若是，就不執行此解，除非符合「Aspiration 機制」。

DecrementTenure()：

更新禁忌期限

- 每次 Tabu Search 主迴圈執行結束時，這個函數要被呼叫一次。
- 所有禁忌中的 Move 期限會減一，過期者會被移除。

[產生鄰居解]

[建立 Move 記錄]

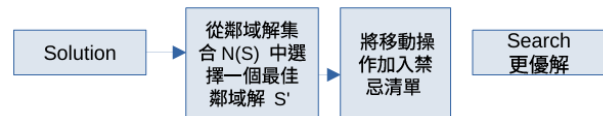
[查詢 Tabu List 是否禁忌]

是 ——→ 排除 / 檢查 Aspiration

否 ——→ 評估 → 選擇最佳解

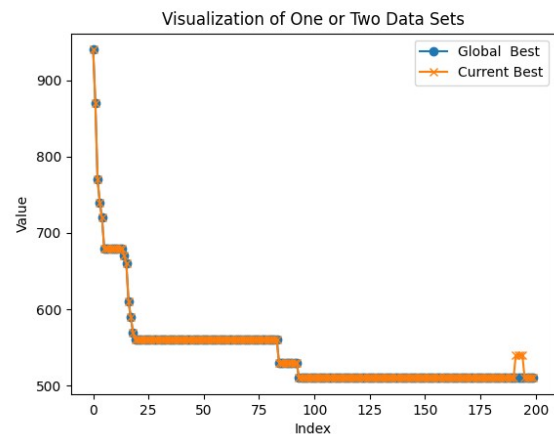
[將選中 Move 加入 Tabu List (add)]

[主迴圈下一輪 → 調用 decrementTenure]



每一次的迭代中，**產生鄰居**，隨機生成多個鄰居（透過交換順序或更換機台）。若移動不在 Tabu List 中，或雖然在其中但可改善歷史最佳（Aspiration），則可被考慮，選出成本最低的可行鄰居，將選中的移動加入 Tabu List，更新目前解與成本，若找到更佳解則更新歷史最佳。每輪結束後，Tabu List 中的移動會減少期限，過期則移除。

Heterogeneity	Best	Worst	Avg
0.0	470	590.	520.
0.2	497.2	539.1	518.04
0.4	407.2	488.8	452.14
0.6	472.8	554.8	510.28



在上述圖表可以看到視覺化後每一次迭代的現行迭代的可行解，解決這項工作匹配排程問題，Tabu Search 已經可以做到很好的結果了，後續也在構思如何更進一步強化 Tabu Search，像是參數設置，引入動態禁忌期限、頻率記錄表，或是多起點多重啟動。

Heuristic

核心思路是用動態就緒清單 (ready list) 的 Kahn 演算法來產生拓撲排序，同時在每一步選擇最能「最快完成」該任務的處理器。

把 `cfg.theTransDataVol` 反向建立「前驅→後繼」清單，方便在完成某任務後更新它的後繼。

- **indegree / ready**：用 Kahn 演算法保證 `ss` 是合法的拓撲序。
- **同時計算 `ms`**：每當選定下一個就緒任務 `t`，就「試算」所有處理器 `p` 上的完成時間，考慮：
- 處理器目前空閒時間 `procFree[p]`
- 所有前驅任務在不同 `processor` 上的結束時間 + 通訊延遲
- 該任務在 `p` 上的計算成本
- **選最快完成**：將 `t` 指派到能最早完成的處理器 `bestProc`，並更新 `startTime[t]`、`endTime[t]`、`procFree[bestProc]`。
- **更新後繼**：將所有依賴 `t` 的任務 `u` 的 `indegree[u]--`，若 `==0` 則加入 `ready`。

N4_00.dag

```
ss : 0  1  2  3  4  5  6  7  8  9
10 11 12 13 14 15 16 17 18 19
ms : 0  0  0  0  0  0  1  0  0  0
0  0  0  0  0  0  0  0  0  0
```

Feasible: true

Cost : 560

N4_02.dag

```
ss : 0  1  2  3  4  5  6  7  8  9
10 11 12 13 14 15 16 17 18 19
ms : 0  3  3  3  3  0  1  3  3  3
3  3  3  3  3  3  3  3  3  0
```

Feasible: true

Cost : 533.8

N4_04.dag

```
ss : 0  1  2  3  4  5  6  7  8  9
10 11 12 13 14 15 16 17 18 19
ms : 0  1  1  1  1  2  1  0  0  0
0  0  1  1  1  1  1  1  1  0
```

Feasible: true

Cost : 522.1

N4_06.dag

```
ss : 0  1  2  3  4  5  6  7  8  9
10 11 12 13 14 15 16 17 18 19
ms : 0  2  2  2  2  1  3  0  0  0
0  0  0  0  0  0  0  0  0  0
```

Feasible: true

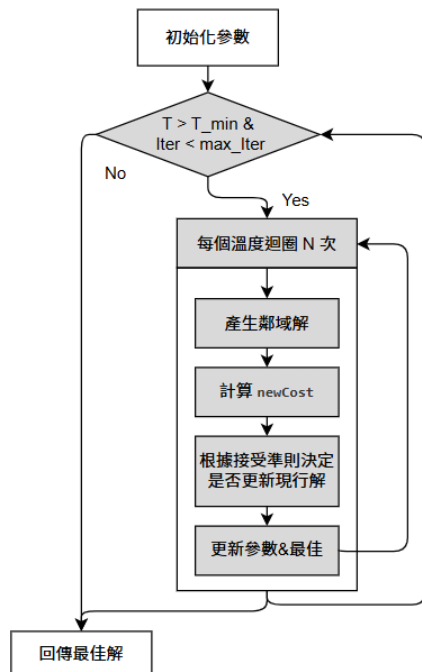
Cost : 524.7

透過 Kahn' s Algorithm 動態生成拓撲排序，再使用清單排程將每一個任務貪婪地分配到最佳處理器。計算每個任務的入度，建構前驅與後繼映射表，初始化所有處理器的空閒時間為 0，可實作為 FIFO 優先佇列) 或隨機挑選來做 Ready List 建立，從 ready list 取出一個任務 `t`，嘗試每個處理器 `p`，所有前驅任務傳輸至該處理器所需通訊延遲與完成時間與處理器可用時間，選擇完成時間最早的處理器，所有任務都被排完，即得到一組合法、可行的解。

後續的使用中可以將其取代隨機解生成，用於區域搜尋，目前我只了解這種啟發式算法可以用於解決排程問題生成一個解出來，我在搜索資訊過程中還看到像是 HEFT 演算法、NEH、Proposed Greedy Topological List Firrst Come First Served(FCFS)、Shortest Job First(SJF)、SRTF、Priority Scheduling 諸如此類的演算法，目前我僅是知道皮毛，無法全部學會用在我的實驗中。

Heuristic + SA

實驗部分：模擬退火與啟發式演算法生成初始解整合在一起，進一步搜尋最佳解，彌補 SA 的搜尋能力上的問題。



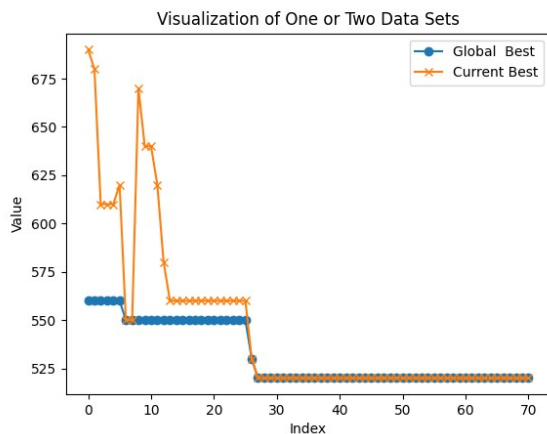
init : 560

Best Solution :

```

ss : 0  1  3  5  7  2  4  9  12  16
     6  15 18 11 17  8 10 13 14 19
ms : 1  0  1  0  0  2  1  0  1  0
     0  3  0  3  0  2  2  2  2  0
  
```

Best Cost : 520 案例 n4_00.dag



00 Avg : 510 04 Avg : 499.368
02 Avg : 499.722 06 Avg : 494.561

Heuristic + TS

實驗部分：Tabu Search 與啟發式演算法生成初始解整合在一起，測試看看已經很優秀的 Tabu Search，是否可以在存在一個好的初始解的情況下有更優秀表現。

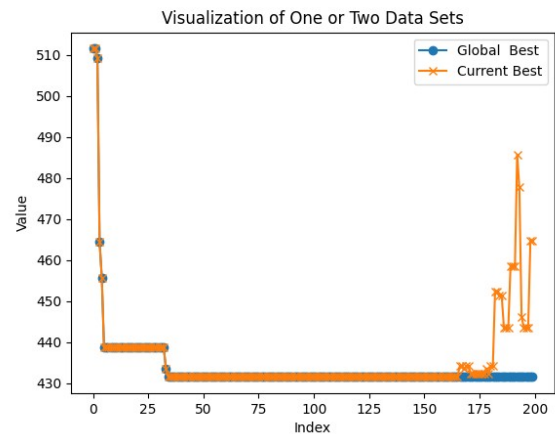
init : 560

```

ss : 0  ,1  ,3  ,6  ,7  ,4  ,2  ,8  ,11
     ,9  ,12 ,13 ,5  ,10 ,14 ,16 ,17 ,15
     ,18 ,19 ,
ms : 3  ,0  ,3  ,0  ,0  ,0  ,1  ,0  ,2
     ,0  ,0  ,1  ,0  ,3  ,0  ,0  ,0  ,0
     ,1  ,
  
```

Feasible: true

Cost : 440



100 次測試最終結果與平均：

Avg Cost = 442.600000

Best Cost = 440.000000

Worst Cost = 460.000000

n4_02.dag : Avg Cost = 434.27

n4_04.dag : Avg Cost = 465.9 (std:0)

n4_06.dag : Avg Cost = 441.5 (std:0)

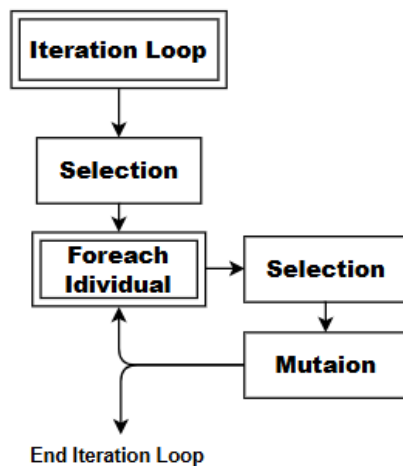
可能很容易陷入局部最優！

這部分測試來說，把 Heuristic 與單一解的元啟發式演算法結合，可以看到效果是有提升的，並且不需要太高的時間成本消耗，因此在測試 SA、Tabu Search 這類進行區域搜尋後，可以有更好更穩定的品質，而元啟發式算法也能夠跳脫區域最佳解。

串聯基因演算法與 Tabu Search

基因演算法與 Tabu Search，此種我想嘗試的整合策略，結合全域搜尋與區域探索的強健策略。基因演算法（GA）進行全域解空間探索，保留多樣性、跳出局部最小，Tabu Search（TS）則聚焦於每個候選解的區域精練，強化局部最佳性，並避免陷入週期性搜尋。

只要有好的起始點，Tabu Search 就有更好的表現能力，GA 透過交配、突變產生多樣後代，減少陷入局部最佳，這對此次實驗問題可以大幅度增強多樣性，在做 Tabu Search 就有一個很好的效果。



第一步實驗，我嘗試分段串聯兩個算法，先做 100 次 GA 迭代，最後篩選出最後的 GA 最佳者，然後以 Tabu Search 進行最佳化搜索。

GA cost : 570

Best makespan: 540

```

ss: 0  1  6  4  3  5  7  9  12 10
8  11 14 15  2 13 16 17 18 19
ms: 2  3  1  3  2  1  1  3  0  2  2
1  2  1  2  2  0  0  0  1
  
```

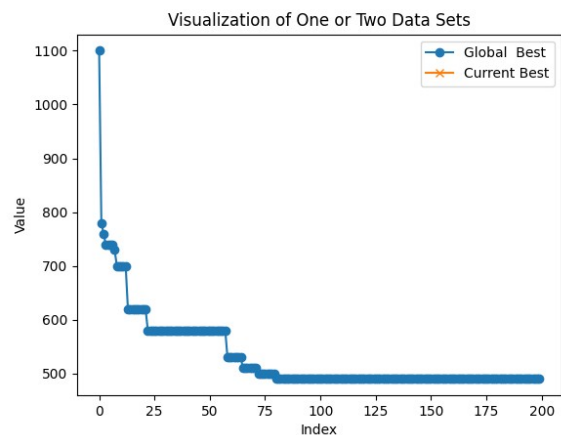
Feasible: true

Cost : 540

Best Cost : 440

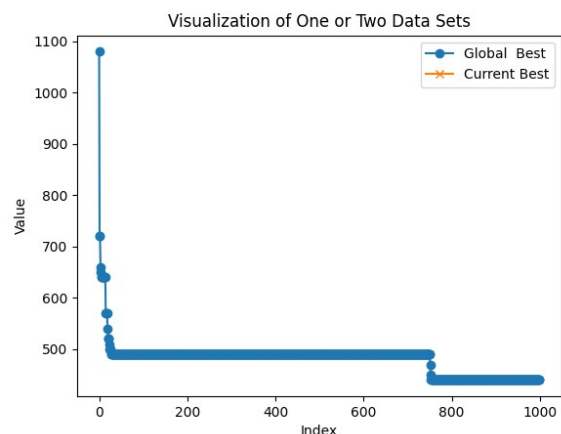
Avg Makespan : 504 (測試 10 次)

迭代：



多次迭代 1000: 各 500

案例 0 -> Cost : 440，其他並沒有很好



跑 100 次

Makespan : 515.6 (n4_00.dag)

Makespan : 505.11 (n4_02.dag)

Makespan : 489.901 (n4_04.dag)

Makespan : 500.45 (n4_06.dag)

我覺得實驗到最後，感覺 GA 100 次 + 串聯 Tabu Search 100 次，可以達到還不錯的效果了，不過似乎還有一些可改進的方向，像是可以取 GA 前 10 者來 Tabu Search 可能可以更好更優秀。後面可以往平行化、多混合加工。

Tabu 突變之基因演算法

在這一次引入的實驗，是想通過融合 Tabu Search 的基因演算法，針對普通 GA 方法，之前測試為競爭式效果最好，第一代世代輪替方法，此處我針對 GA 的變異部分突變函數進行調整，隨機選擇兩個位置交換 ss 基因，或重新指派某個 ms 機台，若個體經突變後發生變化，則以機率 p 進行 Tabu Search 強化。若個體突變後被改變 (ss 或 ms 有動)，則：

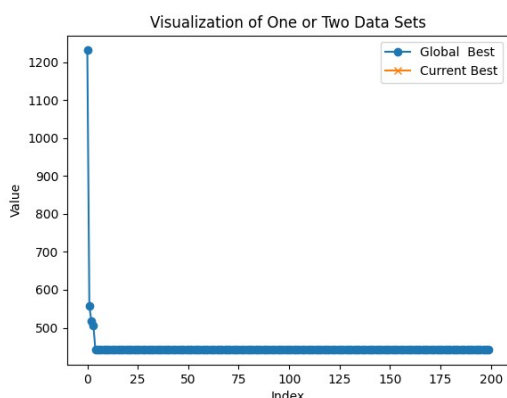
- 以機率 `tabu_probability` 進行 Tabu Search。
- 若未觸發 Tabu Search，則保留突變後個體。
- 若觸發 Tabu Search，則以其結果取代原個體。

`tabuMaxIter`：最多迭代次數 (10)。

`tabuTenure`：Tabu 表持續時間 (5)。

`tabuCandidates`：每次鄰域解候選數量 (20)。

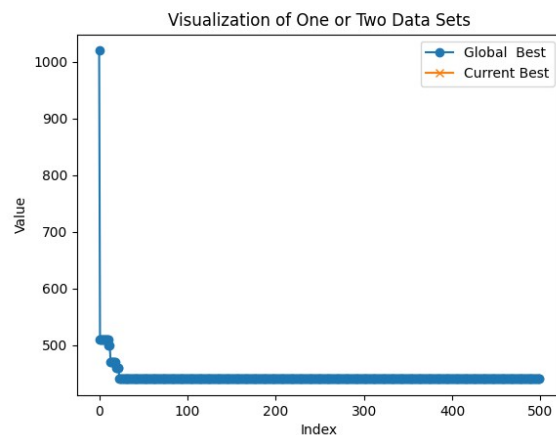
200 Gen 20 Pop



5 次實驗平均 (速度略慢, 1s 左右)

n4_00: 470
n4_02: 452.52
n4_04: 440.08
n4_06: 445.62

500 Gen. 50 pop.



5 次實驗平均 (速度更略慢 5 秒)

n4_00: 450
n4_02: 443.12
n4_04: 433.48
n4_06: 440.46

優勢：可以搜索到品質非常優秀的解

主要問題：太慢...

能力分析上，結合全域探索與局部精煉能力，彈性設計參數，可針對不同問題調整。我覺得未來可以更大更多擴展，根據世代數、族群多樣性自動調整

crossover/mutation/tabu 機率，提升效率與穩定性。由於品質很好所以我覺得也有很多可以加強的拓展方向，可以利用多執行緒或 GPU 同時評估個體與進行局部搜尋，加速整個 GA 進行。

項目	傳統 GA	融合 Tabu GA
解品質提升	一般	良好
收斂速度	中等	較快
避免早熟	否	較穩定
演算法複雜度	較低	較高
可擴展性	中	高

以此就可以以 Hybrid Meta Heuristic 方法來求解此問題，並且結論上來說求解品質非常好。

FOX: A Fox-inspired Optimization Algorithm

論文摘錄：

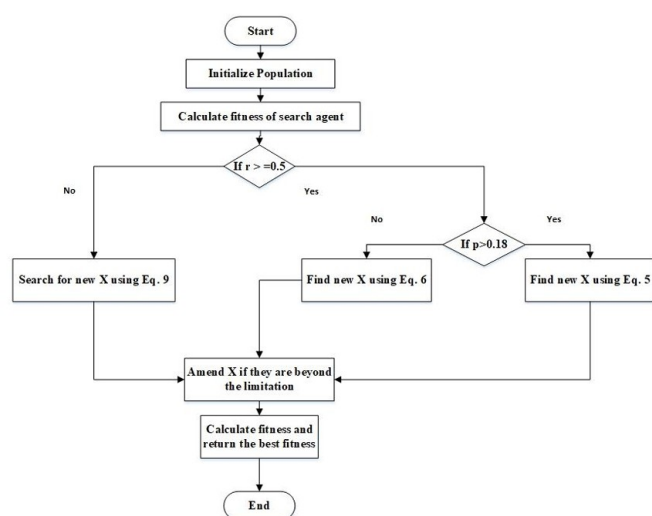
本文提出一種新穎的自然啟發式演算法——FOX 優化器，模擬紅狐在雪地中傾聽聲波、計算距離並跳躍捕獵的行為。結果顯示 FOX 在大多數情況下皆有優越表現，狐狸優化演算法是一個連續型的優化器。

隨機探索 (Exploration)

- 雪地遮蔽視線，紅狐以隨機漫步方式搜索獵物。

聲波定位與跳躍 (Exploitation)

- 紅狐藉由聆聽獵物發出的超聲波，根據時間差計算與獵物的距離；
- 再依計算的距離與跳躍方位估算跳躍高度與步幅

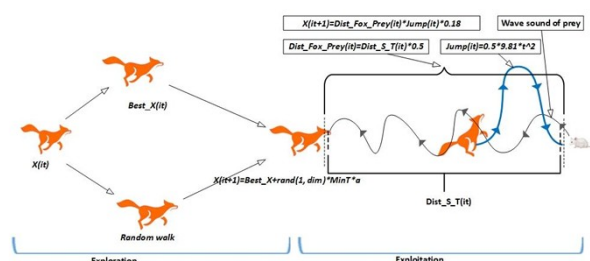


這是一個基於自然生物而啟發的元啟發式演算法，其類似於 PSO，每一次的迭代中 計算每個 $X(i)$ 的適應值，更新 $BestX$ 與 $BestFitness$ ，產生 r ，若 $r \geq 0.5 \rightarrow$ 利用階段，否則 $\rightarrow$ 探索階段。

利用階段：產生 p ，若 $p > 0.18 \rightarrow$ 東北跳躍 ($c1=0.18$)；否則 $\rightarrow$ 反向跳躍 ($c2=0.82$)
- 隨機產生 $Time_ST \rightarrow$ 計算 $Dist_ST$ 、 Sp_S 、 $Dist_Fox_Prey \rightarrow Jump \rightarrow$ 更新 $X(i)$

探索階段：計算 $MinT$ 與 $a \rightarrow X(i) = BestX \cdot rand(1,d) \cdot MinT \cdot a$

以此方式進行優化，簡短來說，每一隻狐狸在每一次的迭代，它們的位置解，會根據獵物進行變動，基於這些狐狸與聲音傳遞，計算與獵物位置，進行解的變動，獵物通常是表示全域最佳解，並且在每一次迭代，遍歷每一隻狐狸，狐狸以 p 機率，決策這隻狐狸改動的模式，分為探索與開發，探索階段以數學模型進行走動，探索更全域的解空間，開發階段，由聆聽獵物發出的超聲波，根據時間差計算與獵物的距離，並以機率分為乘上更大參數，參考最佳解位置去進行跳動狩獵，來靠近最佳解進行收斂。



在使用 SA、GA、TS 等算法行處理求解最佳解問題後，我想開始嘗試去接觸一些其他的演算法，探索新知識、新方法，我便搜索到一些近幾年提出的新興演算法可以來學習並能夠在問題、現實世界求解一些最佳化問題，諸如 狐狸優化、灰狼優化、烏鴉優化等等演算法，蒐集多個論文資料，我也得到了一些學習與實作啟發興趣，狐狸優化演算法由 2022 提出，在那之前也有根據狐狸躲避行為所設計的最佳化元啟發式演算法，在今年 2025 期間，我也看到像是 Improve FOX、Modified FOX 等等更進階的狐狸優化，這進一步的激發我的興趣，讓我很希望不只是在這次的專題實驗跟課堂學習元啟發式演算法，更希望在後續學習更多做法實作演算法解決實際問題，尤其是 FOX 優化器演算法。

FOX 優化演算法實驗

為實驗這個演算法，完成論文上提供的做法與 Github 上開源部分參考，我嘗試將 FOX、FOA 應用在此問題專題實驗上，由於此演算法是為了解決連續空間的問題，一開始就被設計成連續型空間的優化演算法，但排程問題是離散空間的解空間探索與開發，為了用在這個問題上使用連續型演算法，此處我使用，隨機生成浮點數的 N 個解，即狐狸位置。並在每一次的呼叫成本函數計算適存值或 Cost 時進行映射，雙重空間映射，連續到離散解，讓狐狸移動可用向量運算，利用排序大小映射排序字串並利用等頻分組方法映射至離散的解空間中，每代都有「探索」（往 global best）與「開發」（微調、跨足）。

狐狸→Agent 帶有連續位置與速度
獵物→排程最優解的全域最佳位置
跳躍與擾動 對應局部 vs 全域更新

參數：狐狸個體數、最大迭代次數、探索／開發強度參數、兩種跳躍係數、決定採用 Eq.(5) 或 Eq.(6) 的機率閾值、最小平均時間，用在探索步驟、連續空間上下界，長度 $D = 2 \times \text{TCount}$ 。

開發階段

(update_position_exploitation)

根據機率 $p_threshold$ 決定用「向東北跳」（Eq.5）或「向相反方向微擾」（Eq.6）

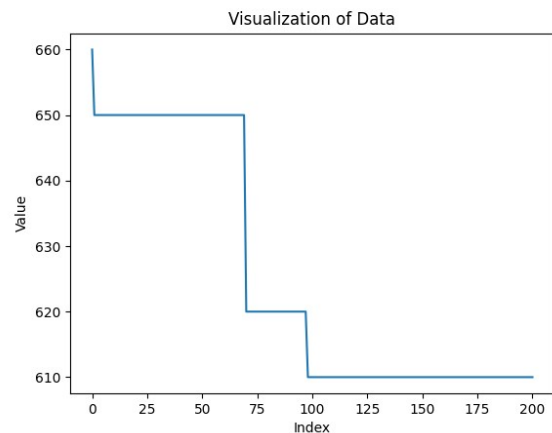
探索階段

(update_position_exploration)

用動態步長與全局最佳 BestX_current 帶動，根據迭代比例計算收斂係數 $a = 2 \cdot (1 - \text{it}/\text{MaxIt})$ ，向全域最佳靠攏、隨機步長擾動。合併後再做「差即回滾」策略。

映射、評估、更新、回滾的迴圈，不斷精煉解，最終回傳高品質的排程 makespan。

FOX 實驗結果



200 pop , 200 Iteration

5 次實驗平均

n4_00 : 580
n4_02 : 574.32
n4_04 : 540.22
n4_06 : 580.38

雖然我花了很多時間閱讀論文並實作此方法，但如上表示，最終結果其實不算很好，大概跟模擬退火差不多，我的想法是這樣，因為整個演算法流程都在連續空間中進行操作，可能導致無法進行模式的捕捉，沒辦法正確的探索跟收斂，演算法在連續空間中更新向量，再透過排序與類別化映射到離散解。此過程常造成鄰近的連續狀態映射為完全不同的排程。若預設值不合問題特性，容易造成過度開發或過度探索。

改進方向：在更新後增設小型 Tabu Search 或局部交換（2-opt）強化步驟，允許接受短期適應度下降再進行更深層優化，打破回滾僵局。在初始化階段引入 HEFT、List Scheduling、或基因演算法生成高品質初解，提供幾隻狐狸更佳種子，提高全域搜尋速度。望提升 FOX 演算法在任務排程問題上的穩定性與最終解品質。

或是直接參考 GWO、PSO 離散版本進行改變演算法更好的能夠用在此類離散問題上，對其數學模型改為離散運算子與離散操作。

鯨魚優化演算法 WOA

鯨魚優化演算法 (Whale Optimization Algorithm, WOA)，在 2016 年提出，靈感源自座頭鯨 (Humpback Whale) 獵食行為中特有的氣泡網喂食 (bubble-net feeding) 策略。該演算法以群體智慧為基礎，模擬座頭鯨在海中捕獵魚群時的三種主要行為：(1) 圍繞獵物游動 (encircling prey)，(2) 螺旋形上升包圍 (spiral-upward motion)，以及 (3) 隨機搜尋獵物 (search for prey)。

在演算法實作上，首先將解空間中的每個候選解視為「鯨魚」的位置向量，並以群體中最優解代表「獵物」的位置。每一次迭代中，對每隻鯨魚依概率選擇執行包圍或螺旋動作。

Encircling Prey

向目標解靠攏 $\vec{D} = |\vec{C} \cdot \vec{X}^*(t) - \vec{X}(t)|$

$$\vec{D} = |\vec{C} \cdot \vec{X}^*(t) - \vec{X}(t)|$$

Search For Prey

全域探索

$$\vec{D} = |\vec{C} \cdot \vec{X}_{\text{rand}}(t) - \vec{X}(t)|$$

$$\vec{X}(t+1) = \vec{X}_{\text{rand}}(t) - \vec{A} \cdot \vec{D},$$

Bubble-Net Attack

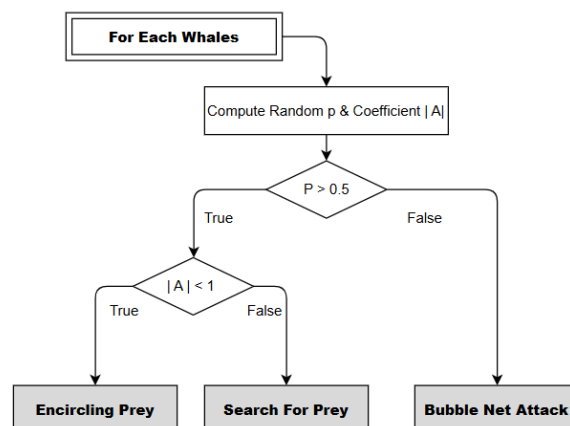
局部搜尋

$$\vec{D}' = |\vec{X}^*(t) - \vec{X}(t)|$$

$$\vec{X}(t+1) = \vec{D}' \cdot e^{bl} \cdot \cos(2\pi l) + \vec{X}^*(t),$$



WOA 透過這三種行為切換，不斷促進全域探索與局部利用之間的平衡，並以目標函數評估解的優劣，最終收斂於全局最優或近似最優解。



根據我先前 FOA 的問題，連續型演算法可能會有出現無法更好搜尋解的問題，儘管原始 WOA 在連續優化問題上展現競爭力，但其設計基於連續搜索空間，故在離散問題上需要進行改造。因此，此次實驗中我想直接採離散結構與離散運算，參考一些資料，多篇文獻提出一些解決方式，直接採用離散結構編碼策略，基於離散編碼、在解序列上直接應用交換、插入、反轉等運算。

Optimizing the Low-Carbon Flexible Job Shop Scheduling Problem with Discrete Whale Optimization Algorithm，基於離散編碼方法重新設計 WOA 來解決這個問題。設計了離散個體更新方法，確保演算法直接作用於離散域。同時，採用混合變鄰域搜尋方法產生高品質的族群。

Green Job Shop Scheduling Problem With Discrete Whale Optimization Algorithm，提出 f1 操作排列採用保優先序交叉 (POX)，速度選擇採用兩點交叉 (TPX)。操作排列採用作業為基礎的交叉 (JBX)，速度選擇採用多點交叉 (MPX)。

參考這些做法我嘗試開始設計一些離散操作運算子來替代鯨魚獵捕獵物的數學模型。

Redesigned Discrete WOA Ver. 1

將經典的 WOA 演算法改寫為完全作用於離散解的版本，以探索其在組合優化場景下的可行性與性能。此實驗的實作演算法是參考離散 WOA 在 FJSP 上的表現一篇提出的作法，並從運算子設計、演化流程與參數敏感度等角度做深入解析，並將此方法嘗試應用在我的專題實驗上。

離散 WOA 同樣保留原論文中三種主要行為：環繞獵物 (Encircling Prey)、搜尋獵物 (Search for Prey) 和螺旋更新 (Spiral Updating)，並用參數 p 與收斂因子 a 控制它們在種群中的出現頻率。具體流程如下：

1. 初始化階段：隨機產生 num_whales 隻鯨魚個體，每隻個體包含一組任務執行順序 (ss) 和機器指派 (ms)，並評估其排程成本 makespan。
2. 迭代階段：對每一代 iter ，先計算收斂因子 $a = 2 \cdot (1 - \text{iter}/\text{max_iter})$ ，並對每隻鯨魚按機率 $p \in [0,1]$ 決定執行哪種更新策略。
3. 最終階段：在完成所有迭代後，回傳全局最優解。

在每次迭代 update 函式中，根據 p 和 a 決定行為：

1. 若 $p < 0.5$ 且 $|a| < 1$ (Encircling Prey)，則使用 $\text{IPOX}(\text{best.ss}, \text{cur.ss})$ 搭配 $\text{MPX}(\text{best.ms}, \text{cur.ms})$ ，即靠近全局最優解進行結構性交叉與指派融合，強化探索最佳區域的能力。
2. 若 $p < 0.5$ 且 $|a| \geq 1$ (Search for Prey)，同樣使用 $\text{IPOX}(\text{randWhale.ss}, \text{cur.ss})$ 搭配 $\text{MPX}(\text{randWhale.ms}, \text{cur.ms})$ ，但融合對象改為隨機鯨魚，保持種群多樣性。
3. 若 $p \geq 0.5$ (Spiral Updating)，則對 ss 用 $\text{ROX}(\text{best.ss}, \text{cur.ss})$ ，對 ms 用 $\text{TPX}(\text{best.ms}, \text{cur.ms})$ ，實現一種更具隨機性和全域性結構重組的螺旋融合。

先後關係基於序列交叉 (IPOX)：

此運算子用於任務順序 (ss) 的更新。先從 parentA 複製前 cut 個基因，再依照 parentB 的出現順序填補剩餘位置，保證子代仍是一個合法排列。此方法兼具父本保留與多樣性創造，避免純隨機交換帶來的結構破壞。

多點交叉 (MPX)

此運算子用於機器指派 (ms) 的更新。對每個任務位置，均以 50% 機率選擇 parentA 或 parentB 的機器編號。多點交叉保持子代與父本在指派上具有混合性，並引入一定隨機性。

隨機基於序列交叉 (ROX)

用於螺旋更新 (Spiral Updating) 中對 ss 的更新。先隨機選取 parentA 中 k 個任務作為前段，之後再依原序補齊剩餘任務。此運算子類似遺傳演算法中的部分映射交叉 (PMX)、順序交叉 (OX)，可在全局結構與局部多樣性之間取得平衡。

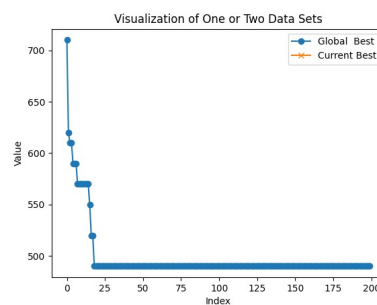
二點交叉 (TPX)

用於螺旋更新中對 ms 的更新。隨機選擇索引 i, j ，然後將 parentB 在 $[i, j]$ 區間內的基因複製到 parentA 對應位置，完成子代。此方式在固定結構中插入少量父本 B 的資訊，使子代既保留 A 的主幹，又吸收 B 的局部優勢。

100 群體、200 迭代

5 次實驗平均

n4_00: 512
n4_02: 527.86
n4_04: 482.76
n4_06: 512.7



Redesigned Discrete WOA Ver. 2

上一個只是基於其他問題而引用來的 DWOA，可能因為設計的太複雜不適用於此問題因此沒有表現得很好，這次我簡單設計一個基於前面實作經驗的離散版鯨魚優化演算法，每個 whale 同時保存一個排程解，建構子鯨魚產生一隨機解。

離散運算子

1. swapSS(Vec &seq)
 - 隨機選取 ss 序列中兩個位置 i , j ，交換它們的值。
2. mutateMS(Vec &ms, int P)
 - 隨機挑一個任務索引 idx ，將它指派到 $0 \dots P-1$ 之間的新機器。
3. prefixCrossover(const Vec &A, const Vec &B)
 - 隨機選一個切點 cut ，先把 parentA 的前 cut 個任務複製到子代，然後依照 parentB 的順序補齊剩餘任務位置，保證排列的合法性且融合兩個父本結構。

更新策略 (update 函式)

整體邏輯根據「機率 p 與收斂因子 a 」決定「探索」或「開發」行為：

探索階段 (Exploration, $p < 0.5$)

子代直接從當前 ss, ms 複製下來，然後呼叫一次 swapSS 和一次 mutateMS。

開發階段 (Exploitation, $p \geq 0.5$)

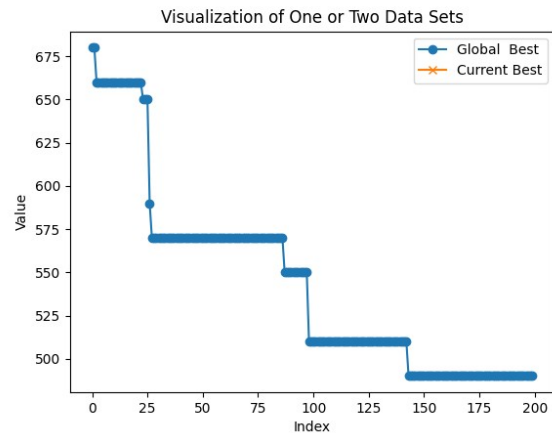
結合全局最優 (best) 與當前先用 prefixCrossover(best.ss, ss) 產生新的任務序列，讓子代向最優解靠攏。

初始 child.ms 先沿用舊指派，再依 $|a| < 1$ 與 $|a| \geq 1$ 做細微微調。

$|a| < 1$ (Encircling Prey) 只呼叫一次 mutateMS，在最優附近做一次小幅局部變異。

$|a| \geq 1$ (Spiral Updating) 呼叫兩次 mutateMS，給予更細緻的指派擾動，模擬螺旋式微調。

種群由若干 Whale 物件組成，每代對每隻鯨魚執行一次，完成全種群更新後，更新全局最優，重複迭代直到達到最大迭代次數。best 代表當前全局最優個體，randWhale 是種群中任意不同個體，用於增加隨機性。



50 Pop, 200 Iteration

5 次實驗平均

n4_00: 494
n4_02: 461.14
n4_04: 453.88
n4_06: 469.7

實驗分析

- 運算子簡單、易於實現，不須映射連續→離散。
- swapSS+mutateMS 兼顧順序與指派的隨機搜尋；prefixCrossover 則兼顧全局結構信息。

此次重新簡易設計離散版本 WOA，透過簡單的離散算子，至於效果其搜尋解品質能力已經與 GA 差不多了。後續可以持續探索更多方法，改進引入交叉保留 MPX、TPX 或是複合離散操作，或整合 SA、TS。

Redesigned Discrete WOA Ver. 3

此次實驗中，更強化離散運算操作，更加更根據鯨魚原先定義的方式與模式還原搜索方法，設計 6 種離散運算操作模式，並在更新時採用對應方式。分成對 SS 處理三種行為操作、對 MS 處理三種行為操作的鯨魚狩獵活動行為。

每次迭代計算收斂因子

$a = 2 * (1 - \text{iter} / \text{max_iter})$ ，

其隨迭代線性從 2 下降到 0。

再透過隨機數 r 建立 $A = 2 * a * r - a$ 。

針對任務順序 (ss)

圍捕 (Encircle) – swapTowardBestSS

這個運算子模擬鯨魚「圍捕」獵物時逐步收斂的行為。給定當前解 ss 與全局最優解 $best.ss$ ，透過參數 A 控制收斂速度：先計算要執行的交換次數 m ，然後隨機挑選 m 對位置 (i, j) ，比較這兩個任務在全局最優序列中的相對位置：若當前順序和最優解的先後關係相反，則交換它們，逐步讓 ss 趨近 $best.ss$ 。

• $m = \text{ceil}(\text{abs}(A) * n / 2)$

螺旋 (Spiral) – twoOptReverseSS

模擬鯨魚沿螺旋軌跡逼近獵物的細緻調整，此運算子選取一段子序列 $[i, j]$ ，將其反轉 (2-Opt)，對排列結構做局部重組，可快速在當前解附近搜索更優路徑。

搜索 (Exploration) blockShuffleSS

當收斂因子過大代表在初期應強化探索，於是透過「截取一段子序列並插入到另一隨機位置」的方式重組 ss ，大幅打亂原有結構並產生新區域解，對高維度排列優化特別有用。

離散運算子設計：針對機器指派 (ms)

圍捕 (Encircle) – greedyAdoptMS

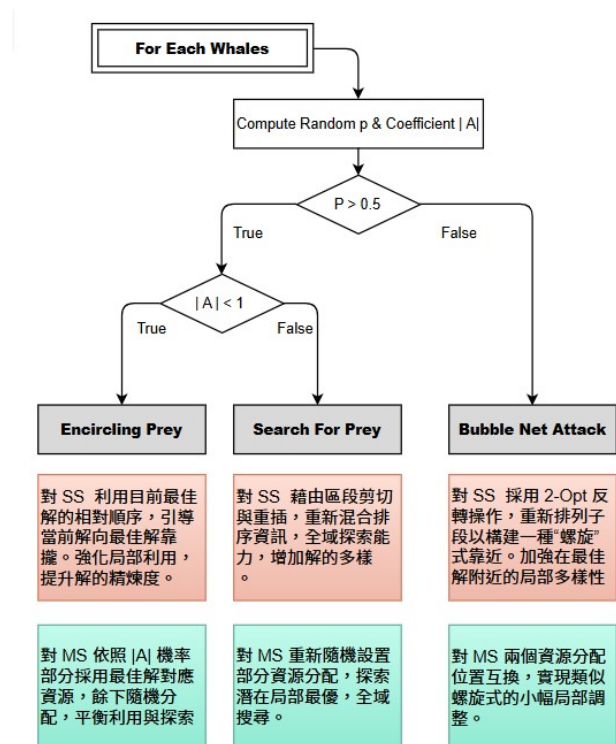
在收斂階段，將當前解的機器指派逐一按照機率 $|A|$ 採用 $best.ms$ 中的對應指派，其餘機率則隨機重設。此舉既能讓多數任務趨向全局最優的指派，又維持少數隨機性以防早熟收斂。

螺旋 (Spiral) – singleSwapMS

用一次「交換兩個機器指派位置」的突變，屬於局部微調，可在指派空間中做輕量級重組。

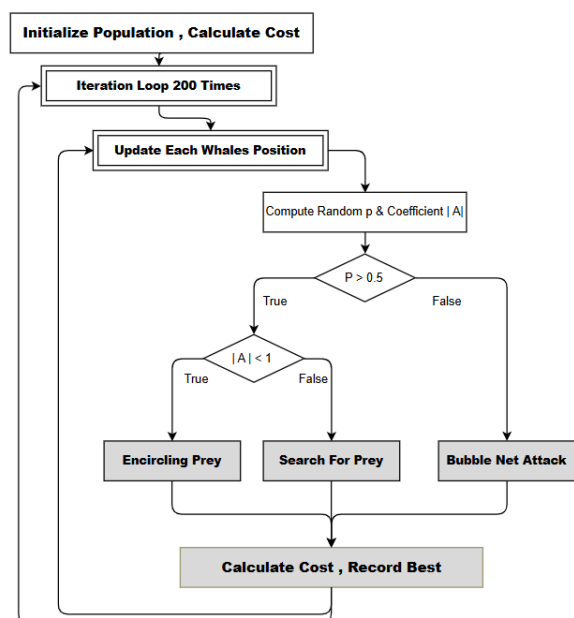
搜索 (Exploration) – randomResetMS

為了增強全域探索能力，此運算子隨機重設約 $\text{ceil}(0.2 * n)$ 個任務的機器指派，使解能跳出指派瓶頸。



演化主迴圈：Whale_Optimize

1. **Initialization**：產生 N 隻 whale，評估初始成本，並記錄全局最佳 **best**。
2. **Iteration**：對每隻鯨魚：
 - 隨機挑另一隻 **randWhale** 以保留種群多樣性（雖未在此版使用，但保留參數）。
 - 計算隨機 p 及收斂因子 A 。
 - 呼叫 **update** 產生 **offspring**，若其成本更優則用之替換當前。
3. **Best Update**：每輪迴圈後掃描整個種群，更新全局最優 **best**。
4. **Termination**：迭代完畢或達到最大迭代次數後，將 **best** 轉為 **Solution** 返回。

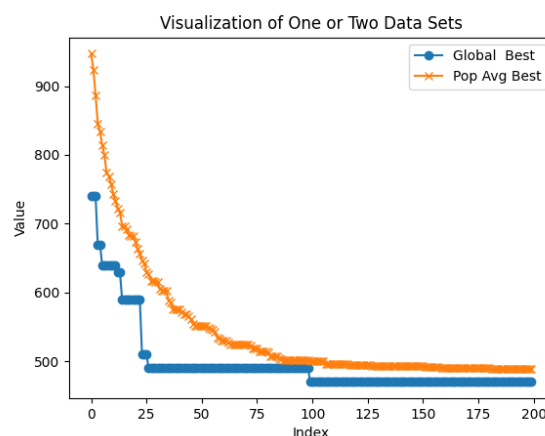


三種運算子（SwapTowardBest、TwoOpt、BlockShuffle）各司其職，分別對應 WOA 的開發、螺旋和探索階段，且都直接作用於排列之上，保證解的合法性。

實驗 DWOA

族群大小：20、迭代次數：200、測試次數：5

Heterogeneity	Best	Worst	Avg	Avg. Time
0.0	470	550	496	171.6 ms
0.2	446.9	520.7	512.14	158.4 ms
0.4	431.9	494.0	469.62	172.4 ms
0.6	481.7	566.7	517.26	155.2 ms



結論分析：

結構與多樣性兼顧：在「圍捕」時段主要靠 **best.ss** 及 **best.ms** 引導，保證種群集中；在「探索」時段打散子序列與重設指派，保留跳脫能力。

收斂—探索平衡：收斂因子 a 與機率 p 共同調控不同階段行為，能逐漸從大範圍搜索過渡到細緻微調，提升收斂速度同時避免早熟。

最終第三版本設計的 **DWOA**，取得了很好的表現，根據實作與統計測試其效果與解品質搜索可以比 GA 還要來的好一點，後續可以再局部強化改進一些部份，或是混合 TS、SA。

在初期廣泛探索多樣解、後期聚焦逼近最優解，最終在排程問題上取得穩定而優良的結果。

DWOA 機率性 Tabu Search 混合演算法

本方法在離散鯨魚優化 Discrete WOA 的基礎上，引入了 Tabu Search 作為局部精煉機制，形成「DWOA-TS」混合策略。演化過程以傳統 WOA 為主幹，通過收斂因子 a 與機率 p 決定每隻鯨魚的更新方式：圍捕（Encircle）、螺旋（Spiral）、探索（Exploration）三種行為交替進行。在每次生成「後代」解之後，以固定機率（例如 8%）呼叫 Tabu Search，將剛產生的候選解作為初始解進行禁忌搜索，進一步在其鄰域內尋找更優解，再將最終結果回寫回 offspring。這種「先全域跳躍再局部精煉」的結合方式，既保留了 WOA 的全域搜尋優勢，又彌補了其對細節調整不足的弱點。

在每隻鯨魚執行完離散運算子更新（swapTowardBestSS、twoOptReverseSS、blockShuffleSS、greedyAdoptMS、singleSwapMS、randomResetMS）之後，若 offspring 已完成初步評估，即呼叫 Tabu Search

```
if (Uniform(0,1) < 0.08)
    Tabu_Search(cfg, &offspring);
```

隨機性決定一個候選解是否進入 Tabu Search 模組，進行鄰域擴展、禁忌列表管理、接受準則判斷等一系列局部搜尋，以期在 offspring 原有基礎上「再優化」。這裡的機率值不宜過高以免頻繁耗費大量 Tabu 運算，亦不宜過低否則難以發揮局部精煉效果，一般設定在 5%–10% 之間較為合適。

混合策略優勢

1. 全域+局部兼容

DWOA 負責全域跳躍式探索，讓種群在大範圍解空間中保持多樣，而 Tabu Search 在少數機率觸發時，專注於放大單點鄰域優勢，快速逼近局部最優。

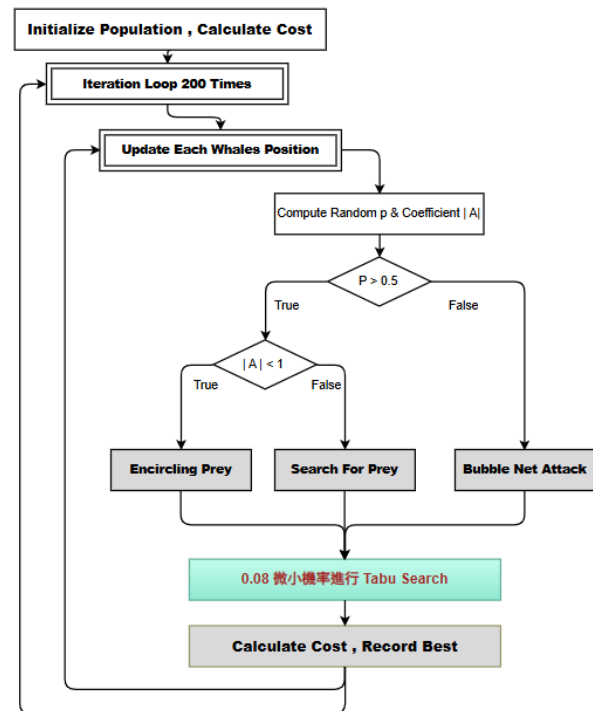
2. 早期多樣、後期精煉

迭代初期收斂因子 a 大，多數鯨魚進行探索或圍捕行為，搭配偶發 Tabu 做小範圍優化；迭代後期 a 小，種群趨於集中，更多鯨魚執行螺旋調整，再以 Tabu 鞏固微小改進，提升精度。

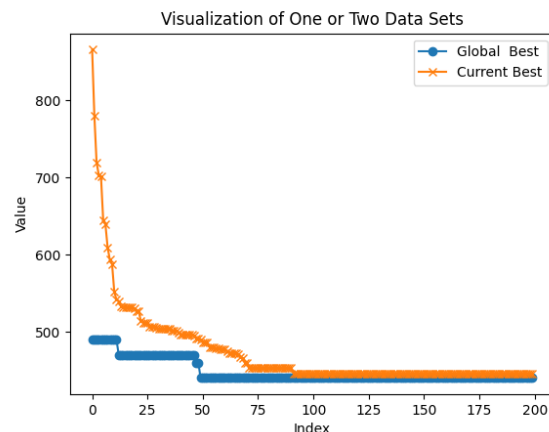
3. 避免早熟與震盪

Tabu Search 的記憶機制能抑制重複返回，減少種群在同一局部來回震盪的概率；同時，若 offspring 因 WOA 更新產生微小逆向變動，也能透過 Tabu 衝破障礙。

演算法架構



實驗結果：



- 案例 0 Avg (5 次) : 440 (5.2 s)
- 案例 2 Avg (5 次) : 425.1 (10.4 s)
- 案例 4 Avg (5 次) : 403.3 (11.2 s)
- 案例 6 Avg (5 次) : 413.8 (10 s)

缺點有一個重大缺陷：

速度太慢計算 5 ~ 10 秒 追求最佳
若優者即可則 2 ~ 4 秒 參數調低

Heterogeneity	Best	Worst	Avg	STD	Avg. Time	Compare Avg	Compare Time
0.0	440	440	440	0.0	9982.2 ms	Fair.	Lose.
0.2	425.1	425.1	425.1	0.0	10981 ms	Win.	Lose.
0.4	403.3	403.3	403.3	0.0	11258.8 ms	Win.	Lose.
0.6	413.8	413.8	413.8	0.0	10040 ms	Win.	Lose.

+10

第一種方法					
Heterogeneity	Best	Worst	Avg.	sd	Avg. Running Time (s)
0.0	440.0	440.0	440.0	0.00	0.4094
0.2	425.1	438.8	427.8	6.12	0.4188
0.4	403.3	426.3	412.3	8.88	0.3968
0.6	423.2	423.2	423.2	0.00	0.4282

Heterogeneity	Best	Worst	Avg	STD	Avg. Time	Compare Avg	Compare Time
0.0	440	440	440	0.0	9982.2 ms	Win.	Lose.
0.2	425.1	425.1	425.1	0.0	10981 ms	Win.	Lose.
0.4	403.3	403.3	403.3	0.0	11258.8 ms	Win.	Lose.
0.6	413.8	413.8	413.8	0.0	10040 ms	Win.	Lose.

+10

第二種方法					
Heterogeneity	Best	Worst	Avg.	sd	Avg. Running Time (s)
0.0	470.0	510.0	492.0	14.83	0.2030
0.2	464.5	500.8	478.4	16.05	0.2030
0.4	448.5	489.6	465.7	16.03	0.2032
0.6	456.8	561.4	499.9	45.67	0.2032

Heterogeneity	Best	Worst	Avg	STD	Avg. Time	Compare Avg	Compare Time
0.0	440	440	440	0.0	9982.2 ms	Win.	Lose.
0.2	425.1	425.1	425.1	0.0	10981 ms	Win.	Lose.
0.4	403.3	403.3	403.3	0.0	11258.8 ms	Win.	Lose.
0.6	413.8	413.8	413.8	0.0	10040 ms	Win.	Lose.

+10

第三種方法					
Heterogeneity	Best	Worst	Avg.	sd	Avg. Running Time (s)
0.0	440.0	460.0	450.0	10.00	1.2372
0.2	425.1	435.2	429.1	4.10	1.6094
0.4	410.8	431.0	421.3	8.23	1.5470
0.6	416.4	435.2	426.2	7.68	1.4342

Heterogeneity	Best	Worst	Avg	STD	Avg. Time	Compare Avg	Compare Time
0.0	440	440	440	0.0	9982.2 ms	Win.	Lose.
0.2	425.1	425.1	425.1	0.0	10981 ms	Fair.	Lose.
0.4	403.3	403.3	403.3	0.0	11258.8 ms	Win.	Lose.
0.6	413.8	413.8	413.8	0.0	10040 ms	Win.	Lose.

+10

第四種方法						
Heterogeneity	Best	Worst	Avg.	sd	Avg. Running Time (s)	
0.0	440.0	460.0	444.0	8.94	0.5218	
0.2	425.1	425.1	425.1	0.00	0.7096	
0.4	403.3	432.5	420.4	14.11	0.6596	
0.6	413.8	436.6	420.2	10.01	0.5688	

Heterogeneity	Best	Worst	Avg	STD	Avg. Time	Compare Avg	Compare Time
0.0	440	440	440	0.0	9982.2 ms	Win.	Lose.
0.2	425.1	425.1	425.1	0.0	10981 ms	Win.	Lose.
0.4	403.3	403.3	403.3	0.0	11258.8 ms	Win.	Lose.
0.6	413.8	413.8	413.8	0.0	10040 ms	Win.	Lose.

+10

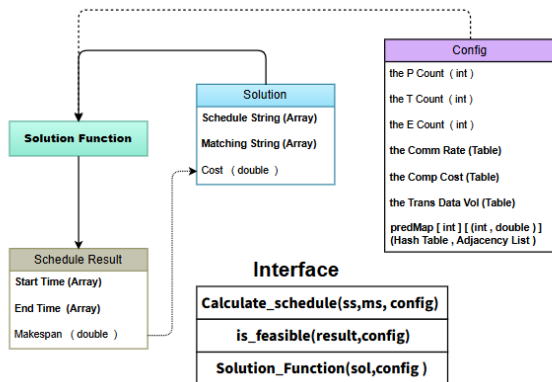
第五種方法						
Heterogeneity	Best	Worst	Avg.	sd	Avg. Running Time (s)	
0.0	520.0	520.0	520.0	0.00	<0.001	
0.2	523.2	523.2	523.2	0.00	<0.001	
0.4	495.7	495.7	495.7	0.00	<0.001	
0.6	455.	455.	455.	0.00	<0.001	

Heterogeneity	Best	Worst	Avg	STD	Avg. Time	Compare Avg	Compare Time
0.0	440	440	440	0.0	9982.2 ms	Win.	Lose.
0.2	425.1	425.1	425.1	0.0	10981 ms	Win.	Lose.
0.4	403.3	403.3	403.3	0.0	11258.8 ms	Win.	Lose.
0.6	413.8	413.8	413.8	0.0	10040 ms	Win.	Lose.

+10

第六種方法						
Heterogeneity	Best	Worst	Avg.	sd	Avg. Running Time (s)	
0.0	470.0	510.0	486.0	21.90	0.9722	
0.2	445.8	474.5	459.8	11.00	1.0096	
0.4	427.9	462.1	447.7	14.20	1.0750	
0.6	413.8	455.7	438.9	22.90	0.7564	

程式模組介紹



專案結構與模組概覽

本專案採用模組化設計，將複雜的排程求解過程拆解為四大核心模組：配置讀取與數據結構（Config & Data）、排程評估與可行性檢查（Evaluation）、工具函式與格式轉換（Utils）、局部搜尋引擎（Tabu Search）／全域演算法入口（DWOA-TS）。以下分別詳細說明前面三大模組的設計理念、介面與數據流程，方便報告書中技術章節的撰寫與演示。

1. 配置讀取與數據結構（Config & Data）

檔案：`config.hpp`

設計理念：

- 分離配置解析與演算法邏輯，確保演算法模組只關注求解過程，而不需處理 I/O 與格式細節。
- 統一使用 Config 結構物件存放所有排程參數與圖結構，並透過 predMap 加速前驅查詢。

主要結構：

- struct Config：
 - thePCount、theTCount、theECount 分別表示處理器、任務與相依邊數。
 - theCommRate：儲存 $P \times P$ 的處理器間通訊速率。

- theCompCost：儲存 $T \times P$ 的任務計算成本。
 - theTransDataVol：列舉每條有向邊的傳輸資料量 [from, to, volume]。
 - predMap：映射每個任務到其所有前驅任務與對應數據量，用於快速計算依賴延遲。
- class Solution：
 - ss：長度為 T 的整數向量，表示排程順序（Permutation）。
 - ms：長度為 T 的整數向量，表示每個任務對應的處理器編號。
 - cost：對應該解的 makespan（排程總完工時間）。

核心 API：

```
Config ReadConfigFile(const std::string &filename);
```

- 支援 .dag、CSV 或自定義格式，透過標籤定位各段資訊。
- 內部以簡易 CSV 解析與 locate_to_section 協助跳過註解。
- 讀完三個矩陣後，建構 predMap，方便後續演算法高效查詢任務前驅。

數據流程：

- 主程式呼叫 ReadConfigFile → 返回 Config cfg。
- cfg.theCommRate、cfg.theCompCost、cfg.predMap 等資料被演算法模組（WOA / Tabu）直接使用。

2. 排程評估與可行性檢查（Evaluation）

檔案：`evaluation.hpp`

設計理念：

- 將排程模擬與合法性檢查抽象為純函式（stateless），方便單元測試與重複呼叫。
- 提供「計算」、「驗證」、「自動修正」三層 API，以因應不同需求：直接模擬、檢

查錯誤，或在求解過程中動態修正不合法解。

核心 API：

```
ScheduleResult Calculate_schedule(
    const std::vector<int> &ss,
    const std::vector<int> &ms,
    const Config &config
);
bool is_feasible(
    const ScheduleResult &result,
    const Config &config,
    bool show_adjust = false
);
ScheduleResult Solution_Function(
    Solution &sol,
    const Config &config,
    bool show_adjust = false
);
```

- **Calculate_schedule：**
 - 根據 ss 指定的任務順序與 ms 的處理器編號，逐步模擬每個任務的 **最早可開始時間**（考慮通訊延遲與前驅完成時間、處理器空閒時間），計算完成時間並回傳 ScheduleResult。
- **is_feasible：**
 - 遍歷所有依賴邊，檢測 $\text{endTime}[\text{from}] \leq \text{startTime}[\text{to}]$ ，若有違反即回傳 false，可選 `show_adjust=true` 輸出錯誤訊息。
- **Solution_Function：**
 - 綜合呼叫 Calculate_schedule 與 is_feasible，
 - 若檢測到違依賴，會 **自動調整** 不合法的任務（移動至序列末端）並重新計算，直至合法。
 - 最後更新 `sol.cost` 為 makespan 並回傳最終調整後的 ScheduleResult。

模組流程：

1. 演算法呼叫 `Solution_Function(sol, cfg, show_adjust)`。

2. 若解已合法，直接返回排程結果。若不合法，以拓譜邏輯排序並返回修正解。
3. 演算法以 `sol.cost` 作為適應度或評估指標。

3. 工具函式與格式轉換 (Utils)

檔案：`utils.hpp`

設計理念：

- 提供跨模組共用的 **格式轉換**、**顯示輸出**、**檔案寫入** 等輔助功能。
- 將繁瑣的 I/O 與格式化程式碼集中管理，保持演算法核心乾淨、聚焦。

子模組：

1. namespace Converter：連續→離散

- `FloatArrayToRankIndex(const vector<double>& arr)：`
 - 將連續值陣列排序，以輸出原始索引對應的順位，常用於 **連續位置 → 任務排列** 映射。
- `FloatToDiscreteClass(const vector<double>& values, int pCount)：`
 - 先排序，再均勻切分為 `pCount` 類別，將連續值映射到處理器編號，實現 **連續位置 → 機器指派**。

2. namespace os_display：結果顯示與檔案輸出

- **顯示工具：**
 - `show_vector`, `show_2d_vector`, `show_solution` 等，快速印出資料結構內容。
- **檔案輸出：**
 - `writeVectorToFile(const vector<double>& data, const string& filename)：`將單一序列寫出，用於繪圖或記錄。

- `writeTwoVectorsToFile(data1, data2, filename)`：同步寫出兩條曲線，用於比較全局與當前演化軌跡。

• 視覺化呼叫:

- `Call_Py_Visual()`：簡單呼叫 Python 腳本 `visual.py`，自動產生收斂曲線圖，便捷整合至報告。

使用範例：

```
// 印出排程解
os_display::show_solution(best);

// 將演化記錄寫檔，並呼叫 Python 繪圖
os_display::writeTwoVectorsToFile(GB_Recorder, CB_Recorder,
"evolution.txt");
os_display::Call_Py_Visual();
```

混合演算法模組 (DWOA + Tabu Search)

檔案：WOA.hpp

設計理念：

- 在離散鯨魚優化 (DWOA) 的全域探索基礎上，**嵌入 Tabu Search** 作為局部精煉機制，以期同時兼顧大範圍搜索與鄰域最佳化。
- 將演化操作 (Encircle、Spiral、Exploration) 與禁忌局部搜尋有機結合，使演算法收斂更穩定且解品質更佳。

核心類別與介面：

- `class Whale : public Solution`：
 - 繼承 `Solution`，封裝 `ss, ms, cost`。
 - 成員函式：
 - `void swapTowardBestSS(Vec &ss, const Vec &best, double A)`：圍捕更新 (Encircle)。
 - `void twoOptReverseSS(Vec`

`&ss)`：螺旋微調 (Spiral)。

- `void blockShuffleSS(Vec &ss)`：探索打散 (Exploration)。
- `void greedyAdoptMS(Vec &ms, const Vec &best, double A, int P)`：指派圍捕。
- `void singleSwapMS(Vec &ms)`、`void randomResetMS(Vec &ms, int P)`：指派螺旋與探索。
- **Whale update(const Whale &best, ..., double a, double p)**：根據收斂因子 `a` 與機率 `p` 選擇適當運算子，並以 **8% 機率** 呼叫 `Tabu_Search(cfg, &offspring)` 進行局部優化。

• 全域演化函式：

```
Solution Whale_Optimize(
    const Config &cfg,
    int num_whales = 20,
    int max_iter = 200,
    vector<double>* GB_Recorder = nullptr,
    vector<double>* PB_Recorder = nullptr
);
```

- 初始化 `num_whales` 隻 `Whale`，找出初代最佳。
- 主迴圈計算收斂因子 `a`，對每隻鯨魚呼叫 `update` 生成 `offspring`，並用 `cost` 判斷是否替換。
- 同時記錄全局最佳 (`GB_Recorder`) 與族群平均成本 (`PB_Recorder`)。

數據流程：

1. Initialization：

GenerateInitialSolution →
Solution_Function 計算初始 cost。

2. Iteration：

- 計算 $a = 2 * (1 - \text{iter} / \text{max_iter})$ 。
- 隨機 p 控制行為分支：圍捕／探索／螺旋。
- 依行為更新 ss, ms → 若 < 8% 機率 → Tabu_Search 精煉。
- Solution_Function 計算 offspring.cost → 替換策略。

3. Record & Return：更新 best，回傳最終 Solution。

組件互動：

ReadConfigFile → Config cfg

→ Whale_Optimize(cfg) → 多次呼叫
update(best, ...)

↳ update 呼叫 WoA 運算子 +
Tabu_Search(cfg, &offspring)

→ Solution_Function(off), 更新 cost

→ 返回最佳解

→ os_display::show_solution &
Call_Py_Visual